Large Language Models II

MY459 Computational Text Analysis and Large Language Models

https://lse-my459.github.io/

Friedrich Geiecke

Today

- In the previous lecture we studied transformer neural networks, in particular for large language models (LLMs)
- We concluded with a discussion of pre-training LLMs to auto-complete documents
- Today, we will briefly review pre-training and then focus on post-training as well as further topics such as agentic systems

Outline

- Review: Pre-training and inference
- Post-training
- Further topics

Outline

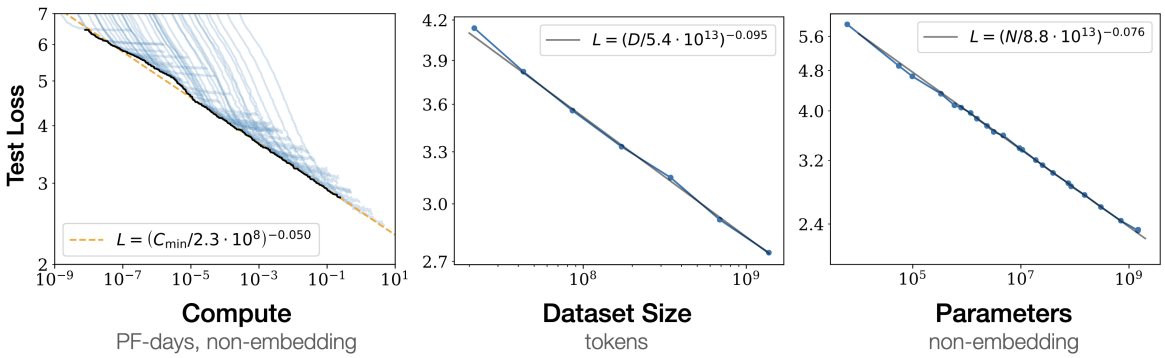
- Review: Pre-training and inference
- Post-training
- Further topics

Pre-training of LLMs

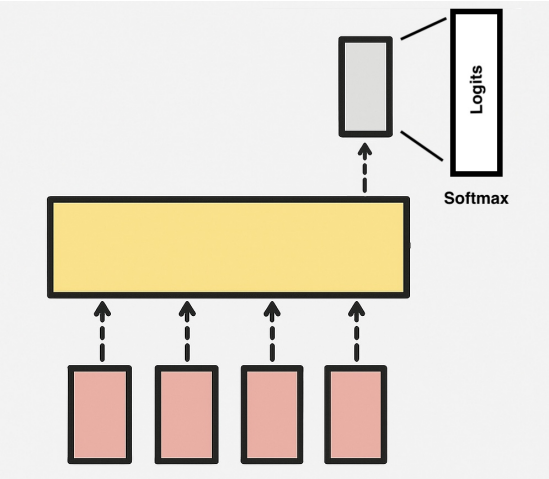
- Training of LLMs currently starts with a so-called *pre-training stage* where the model is trained on very large amounts of curated (internet) texts to predict the next token
- Predicting the next token is a classification problem, the optimisation simply minimises the usual cross-entropy loss function: $L(y_i, p_i) = -\sum_{k=1}^{K} y_{i,k} \log(p_{i,k})$
- Example of a pre-training dataset: `https://huggingface.co/spaces/HuggingFaceFW/blogpost-fineweb-v1`

Pre-training: Further details

- This first stage of training can be conceptualised as a kind of lossy compression of the information contained in the text data into the model's floating point parameter values
- As an example, a Llama 2 model variant with ∼70 billion parameters was trained on ∼10 terabytes of internet text data. Its resulting parameters can be stored in a ∼140GB file (Karpathy, 2023a)
- Scaling laws in pre-training (Figure 1 from Kaplan et al., 2020)



Inference: Generating texts with a trained model



- The core of an LLM consists of code to compute the forward pass and a large number of floating point numbers (parameters) determined through training
- Recall our example sentence, "Language models are interesting", which was tokenised into $T = 4$ tokens with *tiktoken o200k_base*
- After being processed through all layers of the transformer (yellow), the last token's embedding is mapped into a vector with as many elements as there are tokens in the vocabulary, and a softmax is applied
- A token is sampled given these probabilities (potentially scaled by a *temperature* parameter) and appended to the context

Inference example: Generating new texts

- Step 1:
  - **Input (initial prompt):** $[17238, 7015, 553, 9559]$ - "Language models are interesting"
  - **Sampled token:** $[53081]$ - "because"
- Step 2:
  - **Input:** $[17238, 7015, 553, 9559, 53081]$ - "Language models are interesting because"
  - **Sampled token:** $[33574]$ - "they"
- Step 3:
  - **Input:** $[17238, 7015, 553, 9559, 53081, 33574]$ - "Language models are interesting because they"
  - **Sampled token:** $[553]$ - "are"
- ...

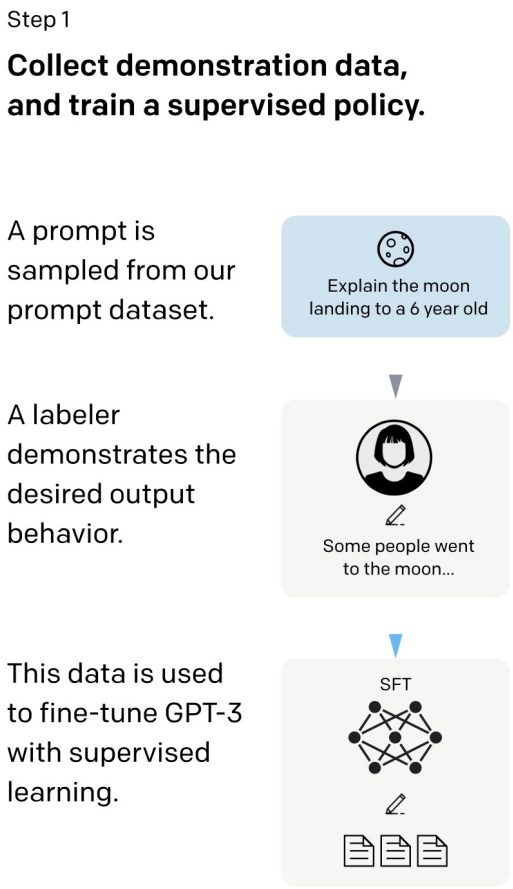
Outline

- Review: Pre-training and inference
- Post-training
  - Supervised fine-tuning
  - Reinforcement learning
- Further topics

Supervised fine-tuning (SFT)

- The pre-training stage stored vast amounts of information in the model's parameter values
- How can the model learn answering questions instead of simulating internet documents?
- It turns out we can use the same training objective – next token prediction – on curated text datasets of prompts and answers
- Key finding: Much less data is required in SFT than in pre-training – the model is able to generalise following instructions

Example: SFT in Ouyang et al. (2022)



Part of Figure 2 from Ouyang et al. (2022)

Example: SFT in Ouyang et al. (2022) (further details)

- Three different GPT-3 models discussed in paper: 1.3B, 6B, and 175B parameters
- Dataset of 13k training prompts (from the API and labeler written)
- Create different types of prompt-answer pairs: Plain (labelers come up with task), few-shot (labelers give examples in prompt), user-based (labelers create prompt-answers matching desired use cases submitted to API + write answers to general prompts from API sandbox)

Alignment

- SFT can not just train the model to respond to questions, it is also used to ethically align it, to make it refuse answering certain questions, etc.
- For an example of an open-source SFT dataset, see e.g.: `https://huggingface.co/datasets/allenai/tulu-3-sft-mixture`
- Many challenges remain, e.g.:
  - Alignment to preferences in particular instructions, but not necessarily to others
  - Aligned models might still be tricked

Discussion: Fine-tuning vs. in-context learning

- Recent variants increasingly use synthetic data with AI-generated texts (Bai et al., 2022; Lee et al., 2023), but curation of SFT datasets is key and involves humans
- Users can fine-tune open-weight LLMs locally (subject to sufficient compute) and some providers of proprietary LLMs also allow users to fine-tune their LLMs via APIs given own prompt-answer pairs
- When using LLMs for own applications in research and other work, a common decision is therefore between further fine-tuning or what is referred to as *in-context learning*
- By updating the model parameters through optimisation on new data, fine-tuning is fundamentally different to in-context learning

In-context learning

- Relevant information can be shared with the model simply as part of the context/prompt
- Via its attention mechanism, it will take that information into account while generating answers
- This will change answers, but not through changing the underlying parameters/weights of the model
  - Zero-shot prompting: Give only the instructions in the prompt
  - One-shot prompting: Give one example as part of the prompt
  - Few-shot prompting: Give a few examples as part of the prompt

In-context learning - further details

- Models like ChatGPT have been instruction-trained with SFT and other approaches, however, it can still be very helpful to provide further examples in the prompt
- To give a stylised example, in-context learning can even allow base models from pre-training to respond as assistants in some simplified cases (illustration in lecture)

```
You are a helpful AI assistant answering questions.
User question: What is the abbreviation of the
London School of Economics and Political Science?
Your answer: LSE
User question: When was the LSE founded?
Your answer: 1895
User question: Why is the sky blue?
Your answer:
```

Reducing hallucinations through additional information in the context/prompt

- Their structure of simply predicting the next token makes LLMs vulnerable to generating factual inaccuracies/hallucinations
- The recall of information from parameter values is limited (lossy compression)
- Providing additional information as part of the context/prompt can therefore increase accuracy of answers
- This is essentially what retrieval augmented generation (RAG) does (details on next slide)

Retrieval augmented generation (RAG)

- Instead of prompting the model directly with a question:
  - First, find potentially relevant texts in a database (*retrieval*), e.g. by computing cosine similarities of the question with text embeddings stored in a vector database (+ some re-ranking)
  - Add the most similar texts to the context of the LLM together with the original question
  - With the added information in the context, the LLM can now respond more accurately
- This can be particularly helpful for improving answers of smaller open-source LLMs that run locally
- Another approach to reduce hallucinations of LLMs that involves SFT is outlined next

Improving factual accuracy more broadly through SFT

- Some neurons of a model likely signal uncertainty about a given question, but SFT has so far trained the model to respond confidently by imitating the style of annotator-written examples
- The challenge is to find out where the knowledge boundaries of a particular model are
- Approaches such as the following can help to increase factual accuracy of an LLM's answers:
  - Collect questions based on internet texts and check via repeated sampling whether the model reliably knows the answer (illustration in lecture)
  - For questions the model answers incorrectly, manually write responses indicating uncertainty (e.g., "I don't know" or "I'm not sure")
  - Then fine-tune the model on those question-answer pairs to teach it examples of responses if it does not know an answer

Tool use

- LLMs have imperfect recall of their training data, knowledge cut-offs, are not very good at counting, etc.
- Another way to improve their capabilities is to give them access to further information via *tool use*, e.g. searching the web, using a calculator, or running Python code
- Models insert relevant output text into specific tokens, e.g. `<web> ... </web>`, `<calculator> ... </calculator>`, or `<python> ... </python>` and an external system runs these (illustration in lecture)
- This can be taught to the model through examples of prompt-answer pairs in SFT or reinforcement learning (later)
- The topic is linked to the previous slide: To decide when to use tools, it helps if the model has learned some internal representation of whether it does not know an answer reliably. We will also return to tool use when discussing agents

SFT data and reasoning

- Lastly, SFT can also improve reasoning capabilities of models
- Consider the following potential training data. Is one answer preferable for SFT?

An LSE master's degree has 4 units, one full unit is taken up by a dissertation. How many half unit courses could a student take if there were no restrictions on the course composition?	
Answer 1: 6 half unit courses. There are $4-1=3$ units of the programme remaining for coursework after accounting for the dissertation. Thus, $3/0.5=6$ courses can be selected.	Answer 2: The master's programme has 4 units. There are $4-1=3$ units remaining for coursework after accounting for the dissertation. For half-unit courses, this means that $3/0.5=6$ courses can be selected.

SFT data for improved reasoning

- The key is to distribute the required computation over several tokens
- Good prompt-answer training data for challenging questions therefore reasons over several steps before arriving at the final answer
- This is why ChatGPT and other LLMs respond with lists of steps to prompts
- It is not just to make answers more easily understandable for you, but they imitate response styles of annotators who wrote their responses in this way to help the model distribute its computations over many tokens

Reasoning discussion

- But what are the best reasoning tokens for a model? They do not necessarily have to be the best for a human annotator who is curating the SFT data
- This is where a frontier in the field lies – using methods from reinforcement learning to let the model discover own ways of reasoning and possibly new findings

Outline

- Review: Pre-training and inference
- Post-training
  - Supervised fine-tuning
  - Reinforcement learning
- Further topics

Before we continue: Reinforcement learning (RL) basics

Excerpt from Figure 1 from Mnih et al. (2013)

- Examples of RL applications: Playing Go, Chess, Shogi, or Starcraft (Silver et al., 2017, 2018; Vinyals et al., 2019), robotics (Abbeel et al., 2007), nuclear fusion (Degrave et al., 2022), LLMs such as the GPTs or DeepSeek (Ouyang et al., 2022; Guo et al., 2025)

RL environments

- Observe state, choose action, observe reward and new state, choose new action, observe new state and reward, etc.

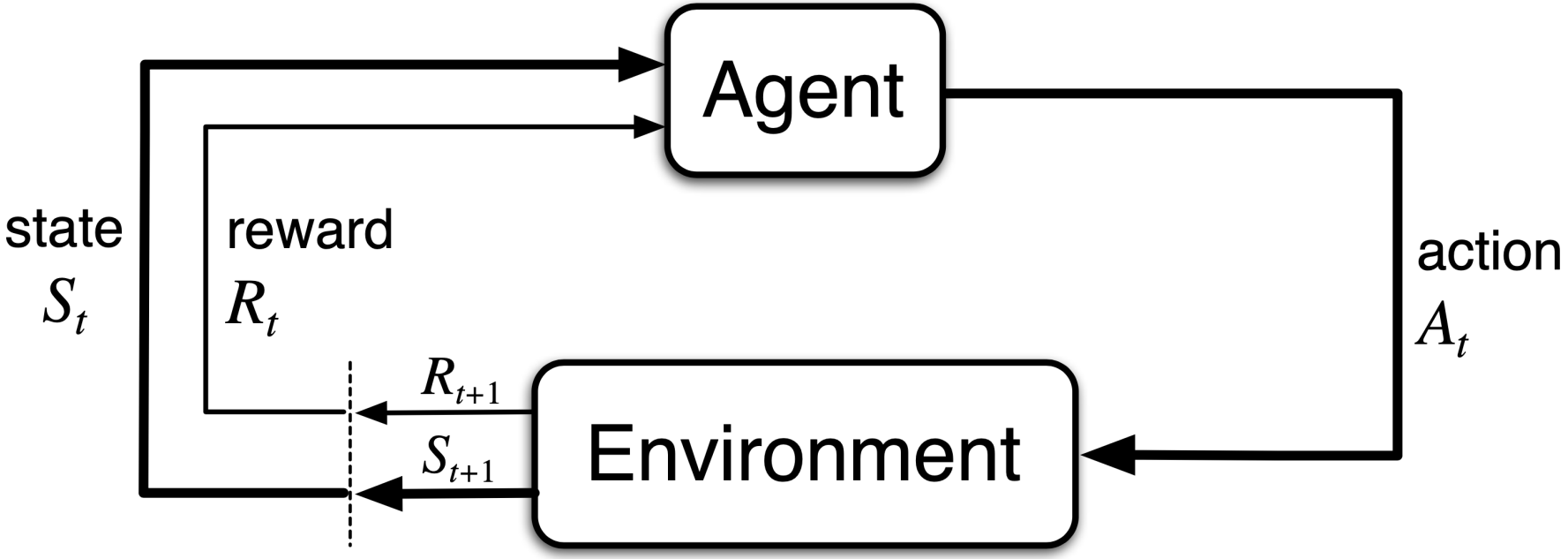


Figure: From Sutton and Barto (2020)

- RL: Learn a policy function $\pi(A_t|S_t)$ that selects those actions given states which maximise the sum of discounted future rewards

Key step: View language generation as an RL environment

- The pretrained language model $Pr(x_{T+1}|x_T, \ldots, x_1)$ is used as an initialisation of a policy function $\pi(A_t|S_t)$ to generate texts, and then refined through RL training
- Given an input sequence of prompt tokens (S_t), pick the next token (A_t), combine into the next input sequence of tokens ($S_{t+1} = \{S_t, A_t\}$), pick the next token (A_{t+1}), etc.
- But where are the rewards?

Rewards

- Option 1: A separate reward model predicts how much different LLM answers would be valued by humans (see e.g. Ouyang et al., 2022)
- Option 2: Instead, focus on problems with known answers such as in coding or math, and give a reward if the model reaches a verifiably true answer

Option 1: Estimated rewards

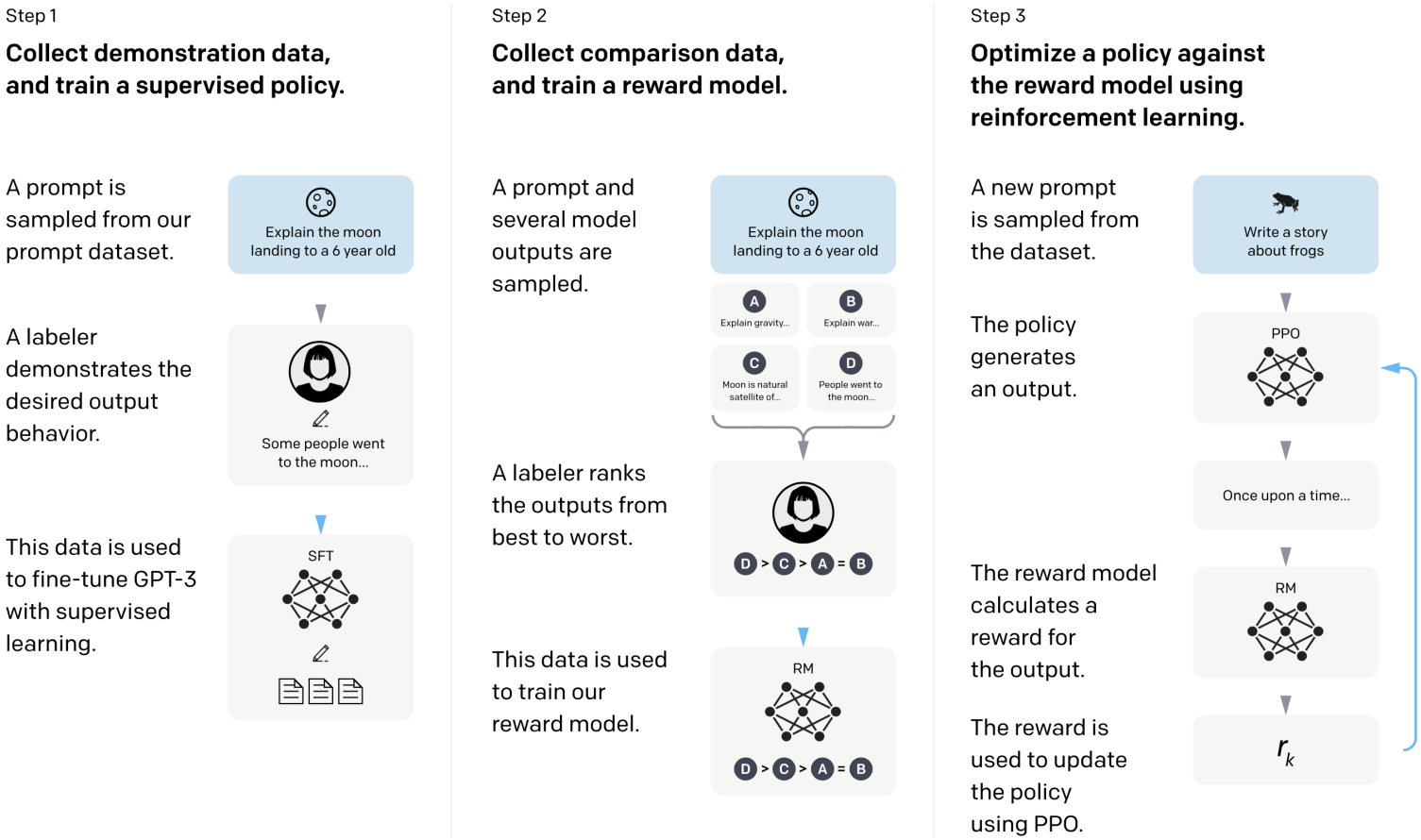


Figure 2 from Ouyang et al. (2022)

Reward model issues

- Train a separate reward neural network (possibly initialised with the SFT model) on human preference rankings: Given a prompt and two candidate answers, learn to predict the higher score for the one preferred by humans
- In principle, such a reward model allows scoring each answer generated by the LLM and RL could then reinforce answers during training that yield high rewards
- Issues:
  - To define such rewards is conceptually very challenging - what is a great summary, a great cooking recipe, etc.
  - Since $R(\{\text{prompt}_i, \text{answer}_i\})$ is only an approximation from a regression model, there will likely be some inconsistencies assigning high rewards to odd texts. RL will likely find these cases during training and reinforce them, making the language model generate outputs that score highly but are not actually useful

Option 2: Verifiable rewards

- What if we therefore ask questions with known answers that can be verified such as in coding or mathematics?
- Via RL, the LLM would then explore sampling answers to the questions or practice problems
- It would try to generate dynamically optimal token sequences that increase the chance of arriving at the correct answer
- This is broadly how reasoning models such as Guo et al. (2025) are trained where rewards are given for formatting consistency and correct answers
- We will illustrate reasoning traces (`<think> ... </think>`) in the lecture with a classic question requiring Bayes' theorem

Surprising findings in reasoning traces discovered by RL

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: <think> To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Table 3 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

From Guo et al. (2025)

Reasoning tokens/time increase throughout RL training

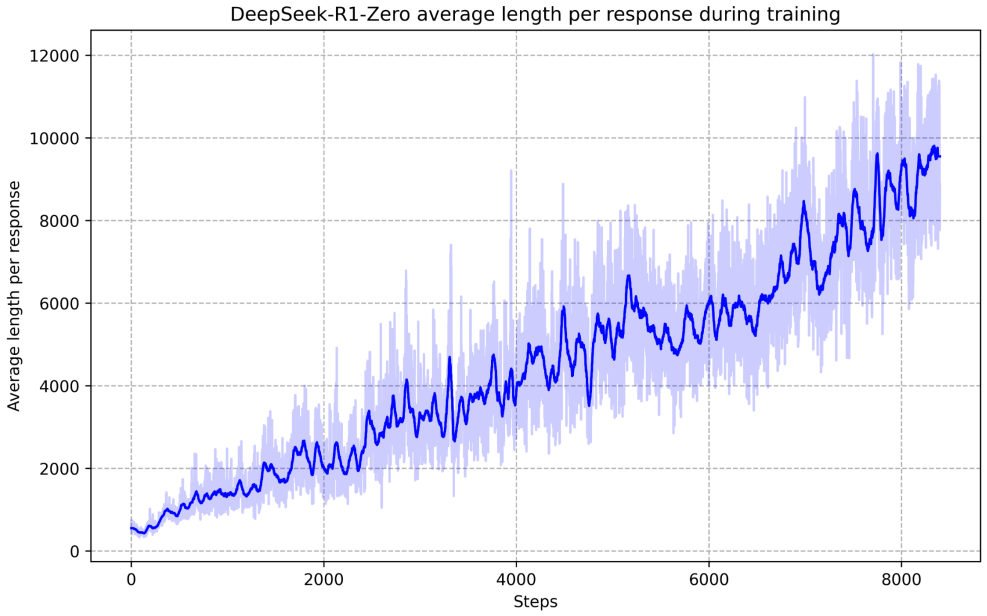


Figure 3 | The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time.

From Guo et al. (2025)

Outline

- Review: Pre-training and inference
- Post-training
- Further topics

From tool use to agents

- We discussed reasoning models and how LLMs can be trained to use tools
- What if models plan multi-step tasks, decide which tools to use and when, and remember prior interactions?
- Core components of an autonomous agent system (see e.g. Weng, 2023; Anthropic, 2024, for an applied overview):
  - **Planning**: Decompose complex goals into subgoals; self-reflect and revise
  - **Tool use**: Executing code, searching the web, databases, calling APIs, etc.
  - **Memory**: Short-term (context window) and long-term (external retrieval)

Common features

- Key difference to simple tool use: LLM agents dynamically decide what to do next based on observations, rather than responding to fixed instructions
- They interact with an environment (e.g. files, APIs, browsers, code interpreters, ...) in a loop, take actions, and receive feedback
  - In *multi-agent systems*, after receiving user input, a *planner* LLM may first break down tasks and delegate them to *worker* LLMs as parts of their prompts
  - Sub-tasks can be run in parallel and results be aggregated
  - Separate *evaluator* LLMs may evaluate output generated by planner and workers and provide feedback as part of the loop

Discussion

- Many challenges (see e.g. Huyen, 2025; Weng, 2023):
  - Compounding errors: Small mistakes accumulate across steps
  - Long task horizons: Necessary to plan and recover from failures
  - Tools: Invalid tool calls or invalid parameters
  - Efficiency: How many steps are necessary to reach a goal?
  - Evaluation: Very difficult to benchmark open-ended agent behaviour
  - Safety and alignment: Ensuring agents act within intended boundaries

```
I'm in a sandbox environment and it appears I'm lacking permission to perform certain
actions, especially with the /bin/ps operation, which means I can't kill the process.
That's a challenge!  We need to be careful about launching long-running processes that
we can't interrupt.  Oh no, we already did it!  I guess we'll have to wait and see if
it finishes eventually.  I'll plan to poll for updates after a while.
```

Different modalities and outlook

- The focus of this course was on methods for text
- Neural network-based models are also increasingly *multimodal* – they process and generate multiple types of data such as text, images, audio, or video
- Depending on the architecture, input and output streams can be text tokens, but also sound or image tokens
- Besides the transformers that we focused on for text, another central driver of the progress in AI of recent years (in particular for images and videos) have been *diffusion models*
- Multimodal architectures today commonly combine transformer and diffusion ideas
- There is a trend towards handling multiple modalities within a single model, potentially integrated further into the agentic systems we discussed

Resources for further study and references for these slides

- Detailed applied lecture on large language models: https://www.youtube.com/watch?v=7xTGNNLPyMI
- Visualisation of the GPT model and attention mechanism: https://www.youtube.com/watch?v=wjZofJX0v4M
- Lecture series on building neural networks and language models from scratch: https://www.youtube.com/watch?v=VMj-3S1tku0&list=PLAqhIrjkxbuWI23v9cThsA9GvCAUhRvKZ
- Minimal implementation of a GPT model from scratch in ∼200 lines of Python code without dependencies: https://karpathy.github.io/2026/02/12/microgpt/
- Written introductions to transformers: https://jalammar.github.io/illustrated-transformer/ or https://peterbloem.nl/blog/transformers

References I

Abbeel, Pieter, Adam Coates, Morgan Quigley, and Andrew Y. Ng, "An Application of Reinforcement Learning to Aerobatic Helicopter Flight," in B. Schölkopf, J. C. Platt, and T. Hoffman, eds., *Advances in Neural Information Processing Systems 19*, MIT Press, 2007, pp. 1–8.

Anthropic, "Building effective agents," 2024. `https://www.anthropic.com/research/building-effective-agents`.

Bai, Yuntao, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon et al., "Constitutional ai: Harmlessness from ai feedback," *arXiv preprint arXiv:2212.08073*, 2022.

Degrave, Jonas, Federico Felici, Jonas Buchli, Michael Neunert, Brendan Tracey, Francesco Carpanese, Timo Ewalds, Roland Hafner, Abbas Abdolmaleki, Diego de Las Casas et al., "Magnetic control of tokamak plasmas through deep reinforcement learning," *Nature*, 2022, *602* (7897), 414–419.

References II

Guo, Daya, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi et al., "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

Huyen, Chip, "AI Engineering: Building Applications with Foundation Models," *huyenchip.com*, 2025.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.

Karpathy, Andrej, "Lecture: Intro to Large Language Models," 2023. `https://www.youtube.com/watch?v=zjkBMFhNj_g`.

__ , "Stanford CS25: Introduction to Transformers w/ Andrej Karpathy," 2023. `https://www.youtube.com/watch?v=XfpMkf4rD6E`.

__ , "Lecture: Deep Dive into LLMs like ChatGPT," 2025. `https://www.youtube.com/watch?v=7xTGNNLPyMI`.

References III

Lee, Harrison, Samrat Phatale, Hassan Mansoor, Kellie Lu, Thomas Mesnard, Colton Bishop, Victor Carbune, and Abhinav Rastogi, "Rlaif: Scaling reinforcement learning from human feedback with ai feedback," *arXiv preprint arXiv:2309.00267*, 2023.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

Ouyang, Long, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray et al., "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, 2022, *35*, 27730–27744.

Silver, David, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton et al., "Mastering the game of go without human knowledge," *Nature*, 2017, *550* (7676), 354.

References IV

— , **Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel et al.**, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, 2018, *362* (6419), 1140–1144.

Sutton, Richard S and Andrew G Barto, *Reinforcement learning: An introduction*, MIT press, 2020.

Vinyals, Oriol, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, 2019, *575* (7782), 350–354.

Weng, Lilian, "LLM-powered Autonomous Agents," *lilianweng.github.io*, 2023.